



# Project Documentation

Online Learning Platform

## Project Links

- GitHub Repository: <https://github.com/MalithGihan/book-learning-platform-gpt>
- Figma Design: <https://www.figma.com/design/kCUcQWpNyD27BLYjfC3Wzq/Book-PT?node-id=0-1&t=azYO3Yf5ln9U6JKs-1>
- Live Site: <https://production.d1vxn4a6u30p6c.amplifyapp.com>

## 1. Project Summary

This project is an online learning platform with:

- User authentication (Register/Login/Logout)
- Role-based access control (student/instructor/admin)
- Course listing + course details
- Course CRUD for instructors/admin
- Student enrollments + “My Enrollments”
- Secure backend practices (cookies, CORS allowlist, rate limit, etc.)

## 2. System Architecture (High-Level)

### 1. Frontend (React + TS + Vite)

- Single Page App (SPA)
- Tailwind UI
- Redux Toolkit + RTK Query for API communication & caching

### 2. Backend (Node.js + Express + TypeScript)

- REST API with ‘/api/v1’ versioning
- JWT auth via HttpOnly cookies (access + refresh approach)
- RBAC middleware to protect instructor/admin routes

### 3. Database (MongoDB Atlas + Mongoose)

- Stores users, courses, and enrollments
- Prevents duplicate enrollments via unique index ‘(studentId + courseId)’

### 3. Frontend Documentation

#### 1. Tech Stack

- React + TypeScript (Vite)
- React Router
- Tailwind CSS
- Redux Toolkit + RTK Query
- Formik + Yup
- lucide-react

#### 2. Frontend Architecture

##### **Pages / Routes:**

- marketing pages, auth pages, dashboard pages

##### **Redux + RTK Query:**

- Central API layer for requests
- Caching + auto-refetch
- Tag invalidation for create/update/delete

##### **UI Components:**

- Reusable layout components (navbar/sidebar)
- Modals/loaders/toasts

##### **Role-based UI:**

- Navigation and available actions depend on user.role

#### 3. Forms & Validation

Formik + Yup:

- Client-side validation
- Inline error messages
- Clean payload submission
- Loading states for submit

#### 4. Frontend Local Setup

- `cd client`
- `npm install`
- `cp .env.example .env`
- `npm run dev`

`# client/.env`

`VITE_API_BASE_URL=http://localhost:4000/api/v1`

## 5. Frontend Deployment

- Set `VITE_API_BASE_URL=https://YOUR-BACKEND-DOMAIN.com/api/v1`
- `npm run build`
- Deploy `dist/` to AWS hosting

## 4. Backend Documentation

### 1. Tech Stack

Node.js + Express (TypeScript)  
MongoDB Atlas + Mongoose  
JWT in HttpOnly Cookies  
Zod validation  
Helmet, CORS allowlist, rate limiting, Origin/Referer guard  
API versioning: `/api/v1`

### 2. Key Backend Features

#### Authentication

- Register, Login, Logout
- GET `/auth/me` returns the current authenticated user

#### RBAC

- student, instructor, admin
- Instructor/admin routes protected (course CRUD)

#### Courses

- Public course listing (published only)
- Role-aware listing:
  - Students/guests: published courses only
  - Instructors: only courses they created
  - Admin: all courses (optional filters)
- Course list adds enrollmentStatus for logged-in students

#### Enrollments

- Students can enroll in a course
- Students can view their enrollments
- Duplicate enrollments prevented (unique index)

## 5. API Endpoints

Base URL: `/api/v1`

### 1. Auth

- POST `/auth/register`
- POST `/auth/login`
- POST `/auth/logout`
- GET `/auth/me`

#### Notes

Auth uses cookies (HttpOnly). Frontend must send requests with credentials enabled.

### 2. Courses

- GET `/courses`  
Public list; for logged-in students it includes `enrollmentStatus`
- GET `/courses/:id`
- POST `/courses` (*instructor/admin*)
- PUT `/courses/:id` (*instructor/admin*)
- DELETE `/courses/:id` (*instructor/admin*)

### 3. Enrollments

- POST `/enrollments/:courseId` (*student/admin*)
- GET `/enrollments/me` (*student/admin*)

### 4. Health

- GET `/health`

## 6. Database Structure (Suggested)

### Users

- email (unique)
- passwordHash
- role (student/instructor/admin)
- Timestamps

### Courses

- title
- description
- category
- level
- tags[]
- price
- instructorId (ref User)
- published (boolean)
- Timestamps

### Enrollments

- studentId (ref User)
- courseId (ref Course)
- status (enrolled/completed/etc. optional)
- timestamps
- Unique index: { studentId, courseId }

## 7. Local Setup (Backend)

- `cd server`
- `npm install`
- `npm run dev`

### ➤ **Server/.env**

- `SERVER_PORT=4000`
- `MONGO_URI=your_mongodb_atlas_uri_with_db_name`
- 
- `CLIENT_ORIGINS=http://localhost:5173`
- 
- `JWT_ACCESS_SECRET=your_secret`
- `JWT_REFRESH_SECRET=your_secret`
- `ACCESS_MINUTES=15`
- `REFRESH_DAYS=7`
- 
- `COOKIE_SECURE=false`
- `COOKIE_SAMESITE=lax`
- `NODE_ENV=development`



## 8. Deployment Guide (Backend + Frontend)

### Backend (Production)

Update server/.env:

- NODE\_ENV=production
- COOKIE\_SECURE=true
- COOKIE\_SAMESITE=none
- 
- # IMPORTANT: include your deployed frontend domain here
- CLIENT\_ORIGINS=https://YOUR-FRONTEND-DOMAIN.com
- 
- MONGO\_URI=your\_production\_mongo\_uri
- JWT\_ACCESS\_SECRET=your\_strong\_secret
- JWT\_REFRESH\_SECRET=your\_strong\_secret
- COOKIE\_SECURE=true
- COOKIE\_SAMESITE=none

### Frontend (Production)

Set:

- VITE\_API\_BASE\_URL=https://YOUR-BACKEND-DOMAIN.com/api/v1

Deploy:

- Build: npm run build
- Host dist/

## 9. Usage Notes

- Users register/login, then role-based UI appears.
- Students can enroll and view enrollments.
- Instructors/admin can manage courses.
- Ensure CORS + cookies are configured correctly for production (HTTPS required for Secure cookies).